

Rakta Bandhan

A Bond of Humanity, Tied by Blood — how the serverless backend finds, alerts, and connects a donor to a request, and what it deliberately keeps private along the way.

Stack Firebase (Firestore · Cloud Functions · FCM · Auth) **Scope** Donor + requester app only

Excludes Admin dashboard & admin* functions

01 SYSTEM DESIGN

Architecture

There is no application server. The Flutter app talks to **Firestore directly**, gated entirely by security rules — the two blocks of logic Firestore can't enforce on its own (geo-matching and access-gated contact reveal) run as **Cloud Functions that react to those same writes**. Nothing in this diagram is aspirational — the dashed edges mark the two paths that exist in code but aren't load-bearing yet.

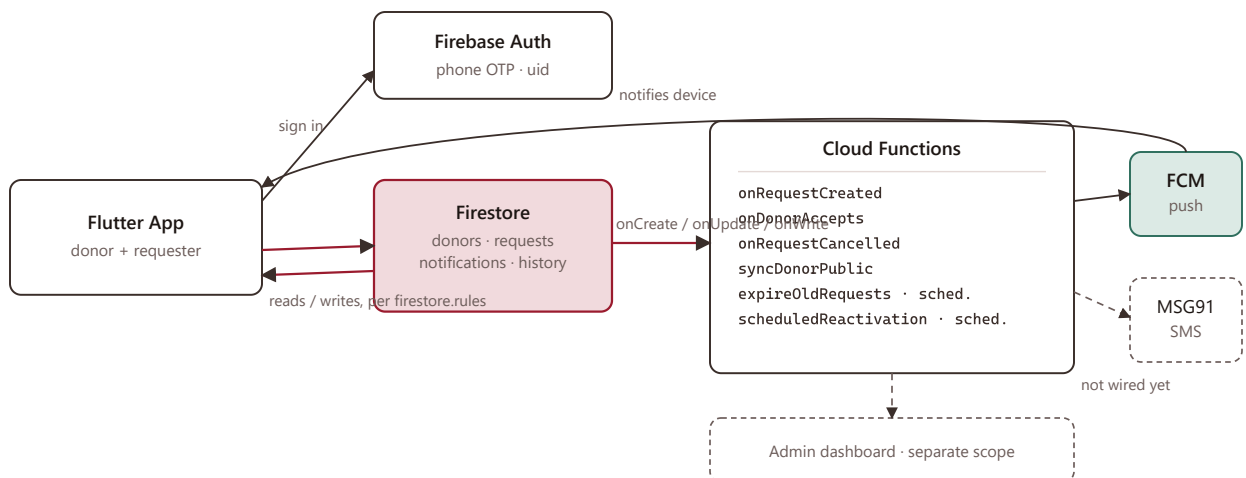


Fig. 1 — The app never calls a REST endpoint; every "API call" is a Firestore write that a trigger reacts to. Only the accent path (Firestore ↔ App ↔ Functions ↔ FCM) does anything today — the two dashed paths are wired in code but not load-bearing.

02 SCHEMA

Data Model

Six collections, one deliberate split. `donors` holds everything, including name and phone; `donors_public` is a Cloud-Function-maintained mirror carrying only what a map pin needs. Nothing in the client ever reads a phone number from either — that field only exists on the one notification document a match created.

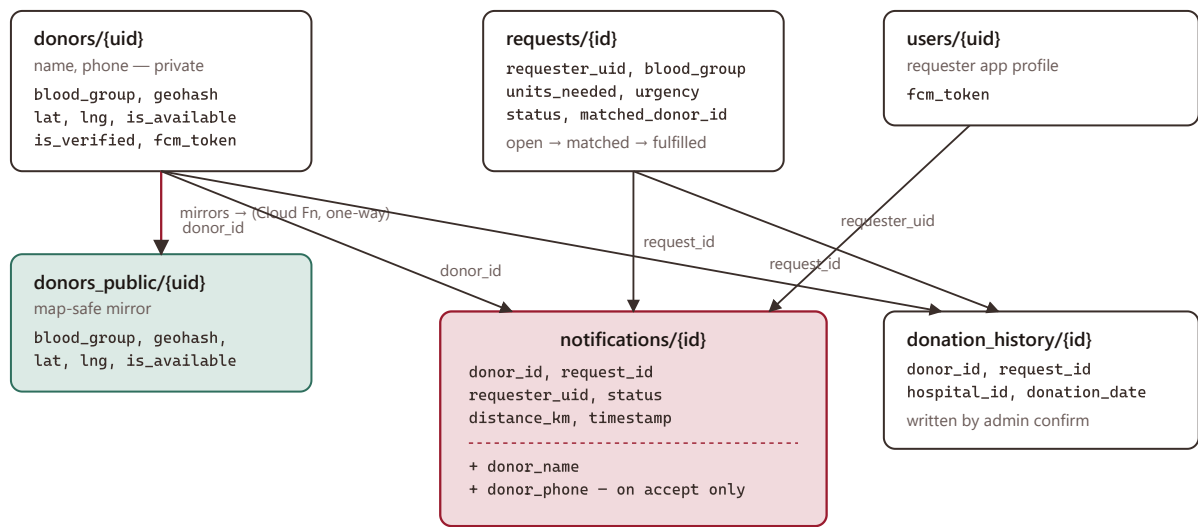


Fig. 2 — `donor_name` / `donor_phone` exist in exactly one place: the specific notification a donor accepted. No collection broadcasts them.

COLLECTION	WRITTEN BY	PURPOSE
donors	Donor (client); Functions on match/donation	Private full profile — name, phone, exact location, verification state.

COLLECTION	WRITTEN BY	PURPOSE
donors_public	Cloud Function only	Map-safe mirror — blood group, geohash, coordinates, availability. No name/phone.
requests	Requester (client); Functions on match/expire	The blood request and its lifecycle status.
notifications	Cloud Functions only	One per donor alerted. Gains contact fields only once accepted.
donation_history	Cloud Functions only	Immutable record of a confirmed donation.
users	Requester (client)	App profile for non-donor accounts.

03 CORE FLOW

Request → Match → Accept

This is the G-I-F-T loop from the initiative brief, as it's actually implemented. Two triggers do all the work; nothing is polled.

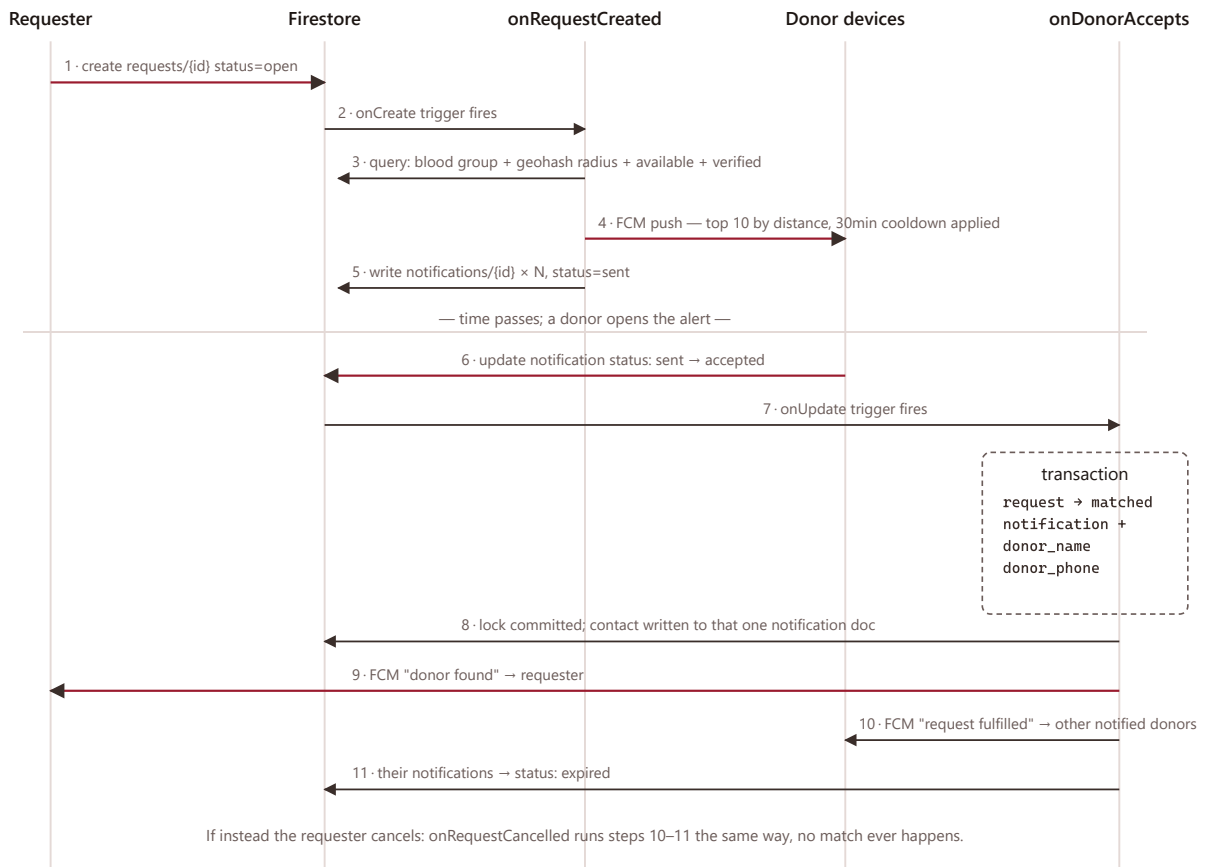


Fig. 3 — The whole flow is two Firestore triggers and a transaction; nothing on the client blocks waiting for a response.

04 TIMING

Donor Availability Lifecycle

The one state a donor can't set themselves is `is_verified` — that's admin-only. Everything else on this line is driven by the donor's own actions or by time.

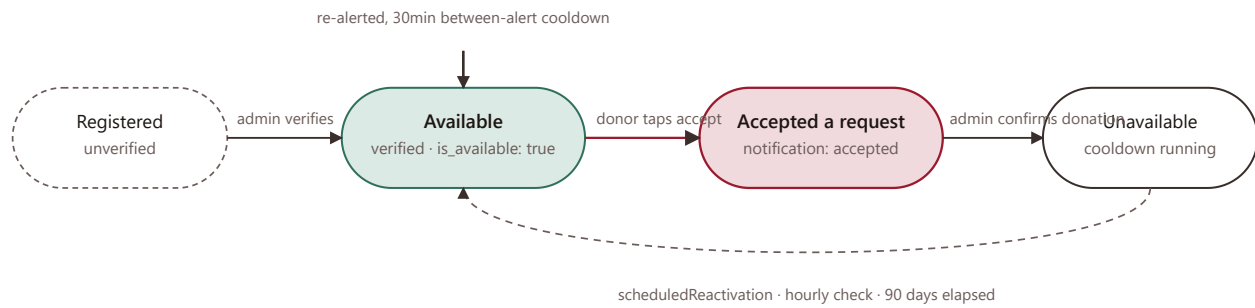


Fig. 4 — The 90-day arc is the one timing constraint a minimal model can't guess right on its own — it's a fixed constant today (`DONOR_COOLDOWN_DAYS`), worth keeping tunable as real donation data comes in.

05 ACCESS DESIGN

Who Can Read a Phone Number

This is the one design decision worth being explicit about, because getting it wrong is exactly how a donor app leaks a phone book. The rule: **never on a broadly-readable document.**

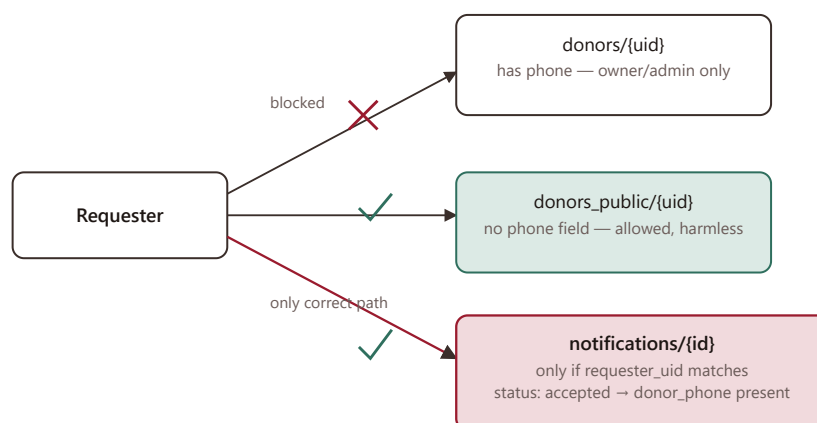


Fig. 5 — Fixed in this pass: the `donors` read used to be open to any verified+available lookup — broadcasting phone and exact location to every signed-in user. It's now owner/admin-only; the map view moved to `donors_public`, which was never given a phone field to leak.

COLLECTION	READ	WRITE
donors	Owner or admin only	Owner creates/updates own profile — can't self-verify
donors_public	Any authenticated user	Cloud Function only
requests	Any authenticated user	Owner creates (open) / cancels; Functions handle the rest
notifications	The donor, the requester, or admin	Donor flips sent→accepted/declined; Functions do the rest
donation_history	Owning donor or admin	Cloud Function only

06 REFERENCE

Function Catalog

Every app-facing function — what fires it, and what it does. fixed this pass marks the three changes made to close the gaps above.

FUNCTION	TRIGGER	DOES
onRequestCreated	Firestore onCreate requests/{id}	Geo + blood-type match, throttle, FCM push, write notifications. crash fixed
onDonorAccepts	Firestore onUpdate notifications/{id}	Transactional lock contact reveal, expire rivals. contact reveal added
onRequestCancelled	Firestore onUpdate requests/{id}	Expire pending notifications, alert donors. new
syncDonorPublic	Firestore onWrite donors/{id}	Mirror safe fields to donors_public. new
expireOldRequests	Scheduled · 30 min	Time out stale open requests.
scheduledReactivation	Scheduled · 60 min	Re-enable donors past 90-day cooldown.

Admin-only functions (onDonationConfirmed , setAdminRole , and the eight admin* callables) run the dashboard and are intentionally out of scope for this document.

For the App Team

→ **Nearby-donors map/list** — query `donors_public`, not `donors`. Same filter fields (blood_group, geohash, is_available, is_verified), no phone number to accidentally render.

→ **Donor contact after a match** — read the matched `notifications/{id}` doc (already what the requester watches to know they've been matched). `donor_name` / `donor_phone` appear there once accepted — nowhere else.

△ **Known gaps, not fixed this pass** — SMS fallback (`utils/sms.js`, MSG91) is written but not called; `MAX_ALERTS_PER_DONOR_PER_DAY` is defined but not enforced, only the 30-minute between-alert cooldown is live.